

merge_segments: Theory, API Guide, and Developer Notes

Mathematical background, aggregation semantics, and implementation guidance

2026-05-18

Table of contents

1	Problem Model	2
2	Mathematical Background	2
2.1	Core reduction formulas	2
2.2	Order-sensitive and categorical reductions	3
2.3	Length-weighted percentile	3
3	Quick Start	4
4	Which API To Use Where	4
4.1	Package-root exports	4
4.2	Module-level API in <code>merge_segments.merge</code>	5
4.3	Recommended selection rules	5
5	Action And Aggregation	6
5.1	Aggregation factories	6
5.2	Deprecated compatibility factories	6
5.3	Numeric-only validation rules	7
6	Visual Guide To Aggregation Behavior	7
6.1	How to read the plots	7
6.2	Case 1: mixed target widths against several source intervals	7
6.3	Case 2: fine target segmentation	9
6.4	Case 3: missing values inside the source stream	9
6.5	Case 4: compact three-by-three example	11
6.6	Case 5: one long target over several source intervals	11
7	Validation And Error Model	13
7.1	Structural requirements	13
7.2	Developer-facing exceptions	13
8	Dispatch, Performance, And Developer Notes	13
8.1	Backend selection	13

8.2	Practical guidance	14
8.3	Performance logger	14
8.4	Environment-sensitive helpers	14
9	Contributor Checklist	15
10	Summary	15

1 Problem Model

`merge_segments` merges interval-valued measurements from a `data` table onto a `target` segmentation. A row in `target` says “produce one output for this interval”. A row in `data` says “this value applies over this interval”. Rows are only compared inside matching `join_left` groups.

For a target interval $T_i = [a_i, b_i)$ and a data interval $D_j = [c_j, d_j)$ in the same join group, the overlap length is

$$\ell_{ij} = \max(0, \min(b_i, d_j) - \max(a_i, c_j)).$$

Only rows with $\ell_{ij} > 0$ contribute to the aggregation for target row i . The library treats intervals as half-open ranges: start inclusive, end exclusive.

The merge is groupwise and rowwise:

1. Partition both frames by `join_left`.
2. For each target row, find overlapping data rows in the same group.
3. Reduce the overlapping values with one aggregation per requested **Action**.
4. Join the aggregated columns back onto the original target rows.

Rows with no overlap produce `NaN` in the derived columns. For each action, `NaN` source values are ignored before the reduction is applied.

2 Mathematical Background

Let v_j be the data value on overlapping row j , ℓ_j the overlap length with the current target row, $L_j = d_j - c_j$ the original data-segment length, and $H = b_i - a_i$ the current target length.

2.1 Core reduction formulas

Aggregation	Definition	Typical use
<code>Average()</code>	$\frac{1}{n} \sum_j v_j$ over non-null overlaps	Simple mean when each overlapping row should count equally
<code>LengthWeightedAverage()</code>	$\frac{\sum_j v_j \ell_j}{\sum_j \ell_j}$	Continuous measurements such as width, roughness, or rate

Aggregation	Definition	Typical use
<code>SumProportionOfData()</code>	$\sum_j v_j \frac{\ell_j}{L_j}$	Totals attached to the source segment length
<code>SumProportionOfTarget()</code>	$\frac{\sum_j v_j \ell_j}{H}$	Rates or percentages normalized to the target length
<code>Sum()</code>	$\sum_j v_j$	Whole-value accumulation when any touch should count fully
<code>Max()</code> / <code>Min()</code>	$\max_j v_j, \min_j v_j$	Numeric envelopes
<code>IndexOfMax()</code> / <code>IndexOfMin()</code>	Index of the row achieving the extreme value	Trace the contributing source row

2.2 Order-sensitive and categorical reductions

Aggregation	Definition	Notes
<code>First()</code>	First overlapping non-null value in input row order	Use when source order is semantically meaningful
<code>KeepLongest()</code>	Choose the distinct value u maximizing $\sum_{j:v_j=u} \ell_j$	Ties are resolved by first appearance in the overlapping rows
<code>KeepLongestSegment()</code>	Choose the value from the single row with largest overlap	Deprecated because it is segment-based, not value-based

`KeepLongest()` is the important distinction in this codebase. It does not ask “which single row overlaps the most?” It asks “which distinct value owns the most total overlap?” If the same category appears on multiple touching rows, their overlap lengths are accumulated before choosing the winner.

2.3 Length-weighted percentile

`LengthWeightedPercentile(p)` sorts the overlapping values ascending, keeps the same overlap weights, constructs cumulative coordinates from adjacent half-weight averages, normalizes those coordinates to $[0, 1]$, and interpolates at percentile p .

For sorted pairs $(v_{(k)}, \ell_{(k)})$, the implementation uses

$$x_1 = 0, \quad x_k = \sum_{r=1}^{k-1} \frac{\ell_{(r)} + \ell_{(r+1)}}{2} \quad \text{for } k > 1,$$

then rescales by x_m and interpolates on the sorted values. This is why the weighted percentile is not simply a cumulative weighted quantile by raw segment length; it is center-point interpolation over ordered segment values.

3 Quick Start

For new code, the most practical entry point is usually `on_slk_intervals(..., legacy=False)`. That preserves the main public function while allowing the fastest available backend.

```
from merge_segments import Action, Aggregation, on_slk_intervals

result = on_slk_intervals(
    target=segmentation,
    data=measurements,
    join_left=["road_no", "carriageway"],
    column_actions=[
        Action("pavement_width", Aggregation.LengthWeightedAverage()),
        Action("surface_type", Aggregation.KeepLongest(), rename="surface_type_majority"),
    ],
    from_to=("slk_from", "slk_to"),
    legacy=False,
)
```

The simple three-target scenario below is rendered live during the Quarto build.

Table 1: Output for the simple three-target scenario used in `test_plot_4`

	road	from	to	keep	LWA	P75	P50	avg	First	prop_sum
0	1	10	20	300.0	400.0	450.0	400.0	400.0	300.0	225.00
1	1	20	30	500.0	500.0	500.0	500.0	500.0	500.0	250.00
2	1	30	40	500.0	450.0	475.0	450.0	450.0	500.0	258.33

Table 1 uses compact labels to keep the PDF layout within the page width: LWA is length-weighted average, P75 and P50 are the length-weighted 75th and 50th percentiles, and `prop_sum` is `SumProportionOfData()`.

4 Which API To Use Where

4.1 Package-root exports

These are available directly from `merge_segments`.

API	Use this when	Notes
Action	You need one output column definition	Combines source column, aggregation, and optional rename
Aggregation	You need to declare how a column should be reduced	Prefer the factory methods instead of constructing enums directly

API	Use this when	Notes
<code>on_slk_intervals</code>	You want the stable top-level entry point	Default is legacy mode; set <code>legacy=False</code> for the fast path
<code>on_slk_intervals_auto</code>	You want environment-controlled dispatch	Reads <code>MERGE_SEGMENTS_DEFAULT_MODE</code> unless <code>prefer_optimized</code> is supplied
<code>on_slk_intervals_optimized</code>	You explicitly want the dense vectorized backend	Good for benchmarking or forcing the non-Numba fast path
<code>on_slk_intervals_numba</code>	You explicitly want the sparse Numba backend	Best raw speed when Numba is installed; fails if unavailable
<code>on_slk_intervals_fallback</code>	You need the categorical-safe backend directly	Rarely needed in application code; used automatically when needed
<code>is_numba_available</code>	You need a feature check before choosing a backend	Useful in diagnostics or deployment checks
<code>configure_performance_logger</code>	You want timing and action-count metrics	Register one callback for performance events
<code>__version__</code>	You need runtime version metadata	Comes from installed package metadata

4.2 Module-level API in `merge_segments.merge`

API	Use this when	Notes
<code>on_slk_intervals_legacy</code>	You need the original row-by-row implementation explicitly	Conservative path, slower, not exported at the package root
<code>AggregationType</code>	You are working on internals	Not the recommended construction surface for user code

4.3 Recommended selection rules

1. Use `on_slk_intervals(..., legacy=False)` for most new application code.
2. Keep `on_slk_intervals(..., legacy=True)` only when preserving long-standing behavior matters more than throughput.
3. Use `on_slk_intervals_auto(...)` when you want deployment-time switching through `MERGE_SEGMENTS_DEFAULT_MODE`.
4. Call `on_slk_intervals_numba(...)` directly only when Numba is a hard requirement and you want failure instead of fallback.
5. Call `on_slk_intervals_fallback(...)` directly only for debugging, regression testing, or explicitly categorical-heavy workflows.

5 Action And Aggregation

An **Action** tells the merge which source column to read, how to aggregate it, and what the output column should be called.

```
Action("pavement_width", Aggregation.LengthWeightedAverage(), rename="width_lwa")
```

5.1 Aggregation factories

The chart suite in the next section visualizes the seven most geometric aggregation behaviors. The remaining selectors and extrema are still important for development work, so they are listed here as well.

All factory names below are methods on **Aggregation**, so the table omits the repeated **Aggregation.** prefix to keep the PDF layout within the page width.

Factory	Output dtype expectation	Use where
KeepLongest()	Any dtype	Dominant categorical or label-valued interval by total overlap
First()	Any dtype	Ordered source streams where first-hit semantics matter
Average()	Numeric	Unweighted mean of overlapping values
LengthWeightedAverage()	Numeric	Continuous measurements averaged by overlap length
LengthWeightedPercentile(p)	Numeric	Upper or lower weighted envelopes without collapsing to min or max
SumProportionOfData()	Numeric	Apportion a source total to the overlapped fraction of the source row
SumProportionOfTarget()	Numeric	Normalize overlap contribution to the target interval length
Sum()	Numeric	Count the whole source value whenever there is any overlap
Max() / Min()	Numeric	Keep only numeric extremes
IndexOfMax() / IndexOfMin()	Numeric	Recover the original source row index for the winning extreme

5.2 Deprecated compatibility factories

Factory	Status	Replacement
KeepLongestSegment()	Deprecated	KeepLongest()
ProportionalSum()	Deprecated alias	SumProportionOfData()

5.3 Numeric-only validation rules

The current validation layer rejects non-numeric source columns for the following aggregations:

- Average
- LengthWeightedAverage
- LengthWeightedPercentile
- SumProportionOfData
- SumProportionOfTarget
- Sum
- IndexOfMax
- IndexOfMin
- Min
- Max

That means string and categorical columns are valid for `KeepLongest()` and `First()`, but not for arithmetic or extrema-based reductions.

6 Visual Guide To Aggregation Behavior

The next figures are rendered by Python during Quarto execution and reproduce the same scenario families as `tests_manual/test_plots/test_plot_1.pdf` through `tests_manual/test_plots/test_plot_5.pdf`. They are the most direct way to see how the major aggregation strategies differ.

6.1 How to read the plots

1. The top-left panel is the original **data** series.
2. Every other panel shows the merged output on the **target** segmentation.
3. Differences between panels are caused only by the aggregation rule, not by a change in source data.

6.2 Case 1: mixed target widths against several source intervals

This is the baseline visual comparison. It shows the difference between unweighted and overlap-weighted reductions clearly.

Interpretation:

- **Keep Longest** follows the value owning the most total overlap within each target row.
- **First** follows source order, so it can differ sharply from both the weighted and unweighted averages.
- **75th Percentile** skews upward relative to **50th Percentile** and the average.
- **Proportional Sum Of Data** accumulates partial contributions from each data segment rather than averaging them.

test_plot_1: render-time reproduction of tests_manual/test_plots/test_plot_1.pdf

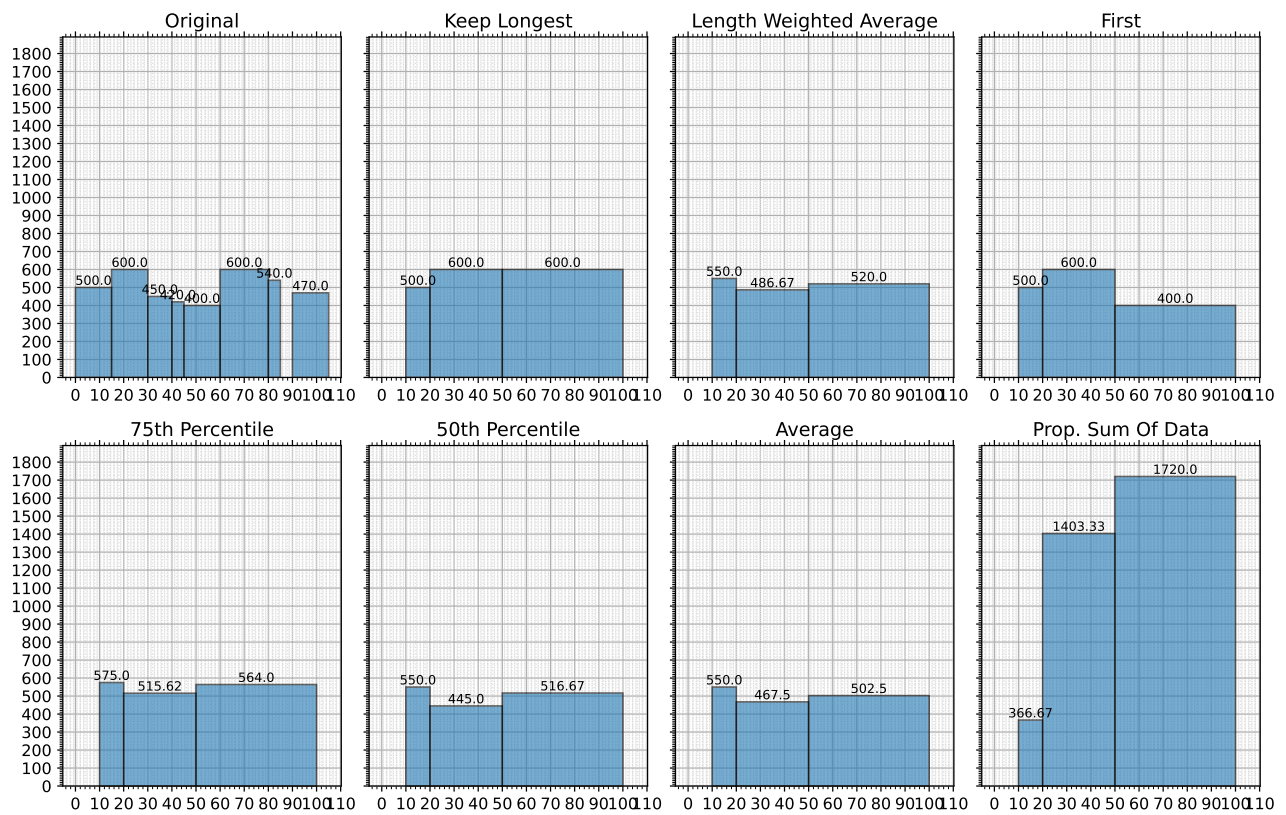


Figure 1: Case 1 reproduces test_plot_1 from tests_manual/test_plots.

6.3 Case 2: fine target segmentation

Finer targets make local transitions obvious and show when **First** changes before or after a dominant weighted result.

test_plot_2: render-time reproduction of tests_manual/test_plots/test_plot_2.pdf

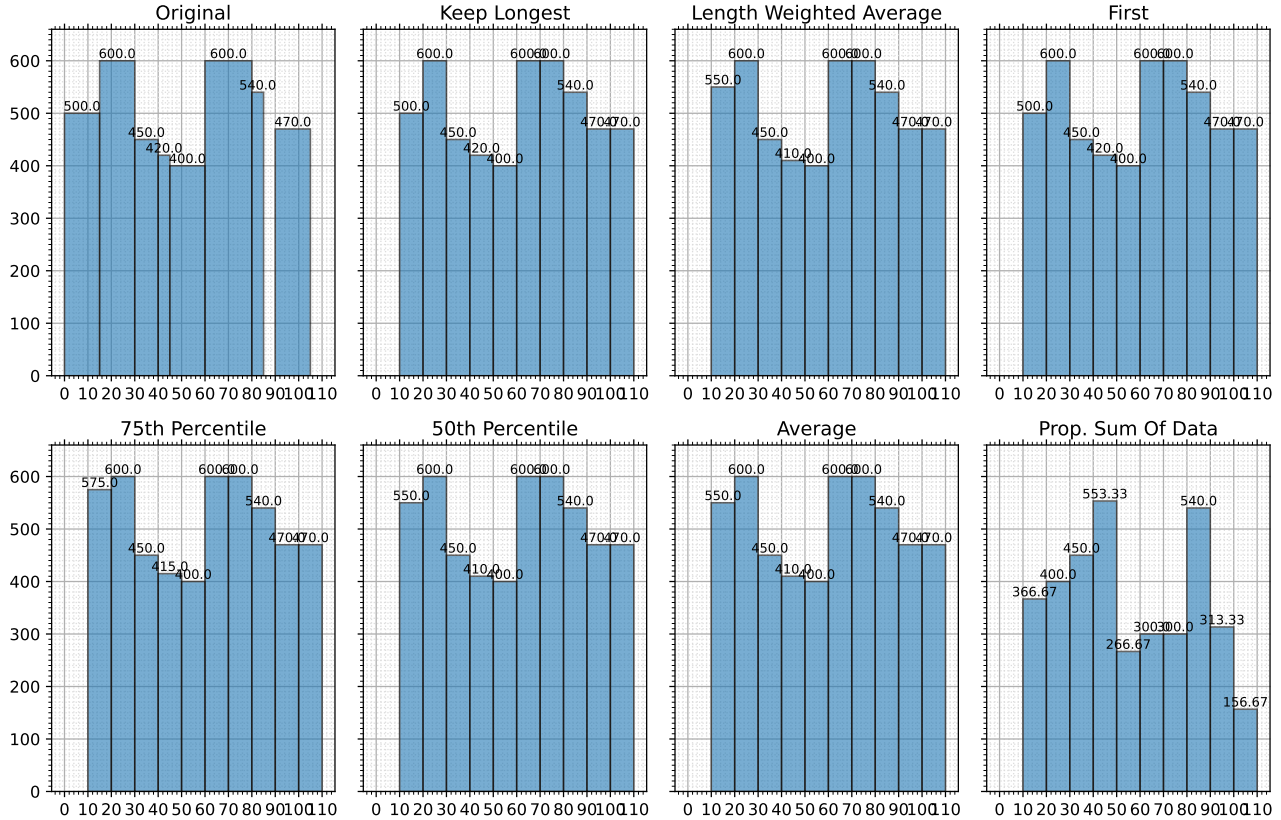


Figure 2: Case 2 reproduces test_plot_2 from tests_manual/test_plots.

Interpretation:

- As target intervals narrow, weighted methods move closer to the underlying raw data values.
- **Average** can still differ from length-weighted methods because each overlapping row counts equally, regardless of overlap size.

6.4 Case 3: missing values inside the source stream

This scenario demonstrates null handling. The **NaN** source row is ignored per action rather than forcing the whole target row to become null.

Interpretation:

- Missing source values are dropped before the reduction is computed.
- The weighted percentile still operates on the remaining sorted values and their overlap lengths.
- A target row becomes **NaN** only when there is no overlapping non-null contribution for that action.

test_plot_3: render-time reproduction of tests_manual/test_plots/test_plot_3.pdf

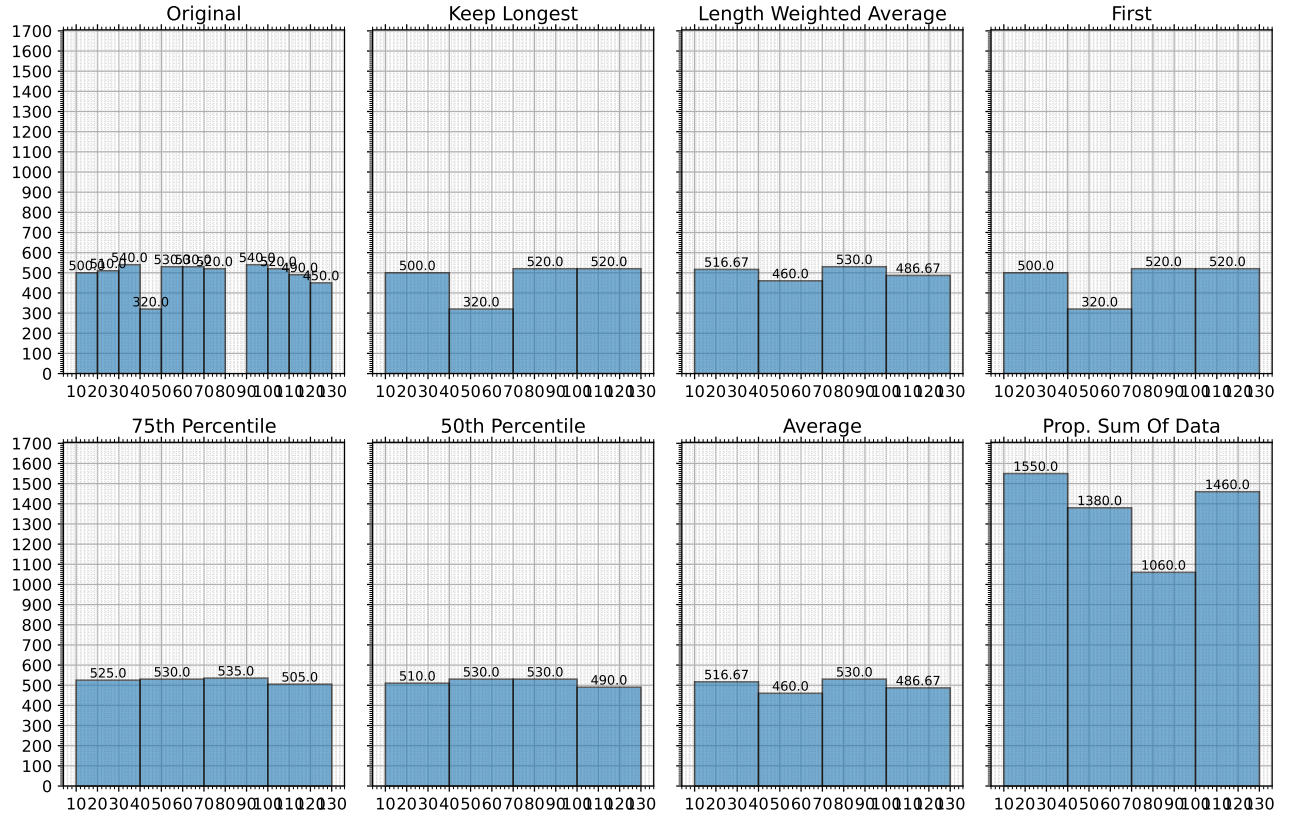


Figure 3: Case 3 reproduces test_plot_3 from tests_manual/test_plots and includes missing source values.

6.5 Case 4: compact three-by-three example

This is the clearest short example for understanding how **First**, **KeepLongest**, and the weighted methods diverge.

test_plot_4: render-time reproduction of tests_manual/test_plots/test_plot_4.pdf

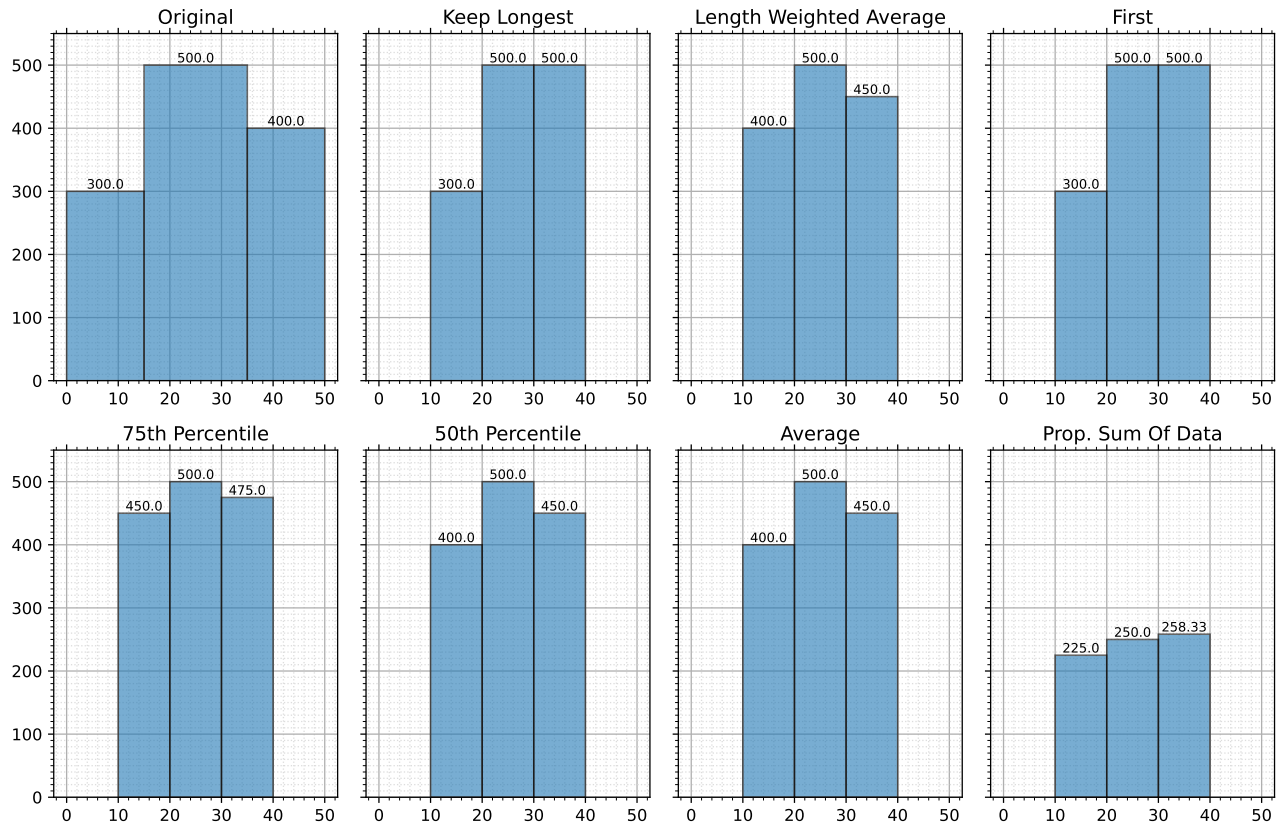


Figure 4: Case 4 reproduces test_plot_4 from tests_manual/test_plots.

Interpretation:

- The center source row dominates overlap in the middle target interval, so **KeepLongest** selects it even when another row touches the target.
- **First** reports whichever overlapping row appears first in source order, not the numerically largest or longest one.

6.6 Case 5: one long target over several source intervals

This case is a good mental model for corridor-level reporting where many shorter observations are merged onto one long management segment.

Interpretation:

- **LengthWeightedAverage** gives a corridor-wide mean.

test_plot_5: render-time reproduction of tests_manual/test_plots/test_plot_5.pdf

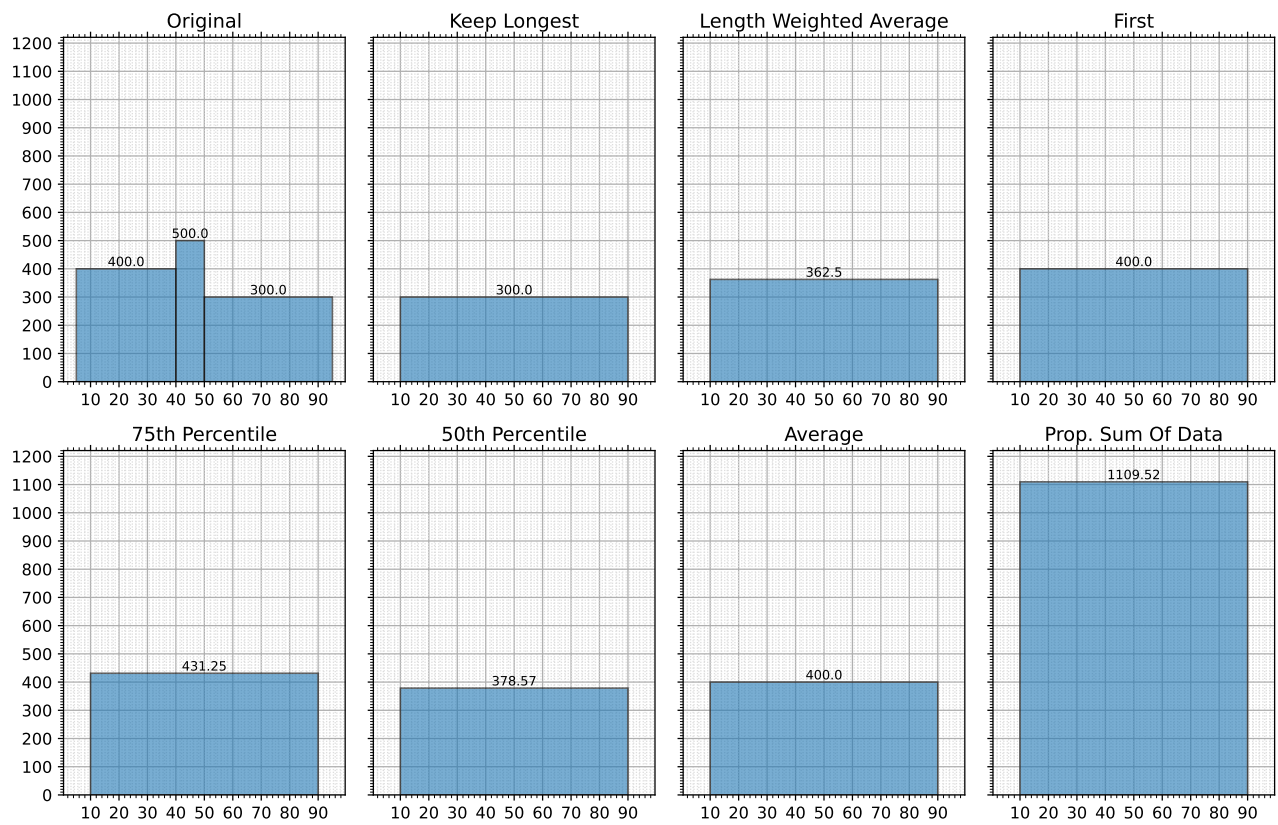


Figure 5: Case 5 reproduces test_plot_5 from tests_manual/test_plots.

- `KeepLongest` reports the dominant categorical state over the corridor.
- `SumProportionOfData` is the right mental model when the source values represent totals attached to the source rows.

7 Validation And Error Model

The public merge helpers share a common validation layer before any expensive overlap work begins.

7.1 Structural requirements

1. `target` and `data` must be pandas DataFrames.
2. Neither frame may use a `MultiIndex` for rows or columns.
3. Indices and column names must be unique.
4. All `join_left` columns and both `from_to` columns must exist in both frames.
5. Zero-length rows where `from == to` are rejected.
6. Requested output names may not collide with existing target columns.
7. Numeric-only aggregations require numeric source dtypes.

7.2 Developer-facing exceptions

Exception	Trigger
<code>InvalidDataFrameError</code>	Non-DataFrame input or unsupported index structure
<code>DuplicateLabelError</code>	Duplicate row index or duplicate column labels
<code>InvalidJoinConfigurationError</code>	Missing <code>join_left</code> or interval columns
<code>ZeroLengthSegmentError</code>	A source or target interval has zero length
<code>OutputCollisionError</code>	An action would create a column that already exists in <code>target</code>
<code>InvalidAggregationError</code>	Aggregation is incompatible with the source dtype
<code>PercentileConfigurationError</code>	Invalid or missing percentile parameter

For contributors, the validation code lives in `_validation.py`, and the named exception classes live in `exceptions.py`. Raising the specific exception types is preferred over generic `Exception` in new code because it keeps debugging and test assertions precise.

8 Dispatch, Performance, And Developer Notes

8.1 Backend selection

The implementation family is now easiest to think of as a dispatcher plus three concrete backends:

1. `on_slk_intervals(..., legacy=True)` routes to the original row-by-row implementation.
2. `on_slk_intervals(..., legacy=False)` routes to the fastest available backend.

3. The fast route prefers `on_slk_intervals_numba()` when Numba is available.
4. Otherwise it uses `on_slk_intervals_optimized()`, which builds dense overlap matrices per join group.
5. If categorical `KeepLongest()` inputs are detected, the optimized path delegates to `on_slk_intervals_fallback()` to preserve stable categorical behavior.

8.2 Practical guidance

- Prefer `legacy=False` for throughput-sensitive production code.
- Prefer `on_slk_intervals_auto()` when you want runtime switching through `MERGE_SEGMENTS_DEFAULT_MODE=` or `legacy`.
- Prefer direct backend calls only in benchmarks, backend-specific tests, or debugging sessions.
- Do not call internal helpers from `_validation.py` outside package development; they are implementation details.

8.3 Performance logger

The library can emit timing metrics through `configure_performance_logger()`.

```
def log_metrics(event: str, metrics: dict[str, float]) -> None:
    print(event, metrics)

configure_performance_logger(log_metrics)
```

For ad hoc diagnostics, `MERGE_SEGMENTS_PERF_LOG=stdout` also enables a built-in stdout logger during process startup.

8.4 Environment-sensitive helpers

- `is_numba_available()` reports whether the Numba backend is available.
- `on_slk_intervals_auto()` consults `MERGE_SEGMENTS_DEFAULT_MODE` when `prefer_optimized` is omitted.

The current render environment reports Numba availability below.

Table 2: Runtime backend availability in the environment used to render this document

	numba_available	recommended_entry_point
0	True	<code>on_slk_intervals(..., legacy=False)</code>

9 Contributor Checklist

1. Add new public behavior through `Action`, `Aggregation`, or one of the documented merge entry points.
2. Extend `_validation.py` when a new invariant should fail early.
3. Prefer explicit exception classes from `exceptions.py` for new failure modes.
4. Benchmark changes against both the legacy and fast paths when modifying overlap logic.
5. Keep categorical behavior stable, especially for `KeepLongest()` tie handling.

10 Summary

Use `on_slk_intervals(..., legacy=False)` as the default modern entry point, choose aggregations based on the physical meaning of the source values, and read the plot suite as a visual reminder that weighted, ordered, categorical, and total-preserving reductions answer different questions. The rendered figures in this document are the same scenario family used by the repository’s manual plot tests, but generated fresh during Quarto execution.